Transaction

1. Feature Overview

Immutable ledger of credits/debits (and later transfers). Foundation is in place under `com.bankone.transaction`: entity, repository, `TransactionService.record`, **CREDIT** on deposit and on opening deposit (`openAccount` when amount > 0, narration “Opening deposit”), and `GET /accounts/{accountId}/transactions` (paged list). No ledger UI yet.

Status: Partial (write on deposit + opening deposit + list API; no withdraw/transfer/UI)

2. Business Purpose

Provide audit-grade money movement history, feed reports/dashboard, enable withdraw/transfer.

3. User Workflow

Implemented paths: Accounts list → Deposit → balance update + CREDIT ledger row; open account / customer create with openingDeposit > 0 → CREDIT with narration “Opening deposit”.

API ready for Account detail → transaction list. Still planned: withdraw/transfer create rows; sidebar Transactions route (nav stub today); ledger UI.

4. Execution Flow

List by account (implemented):

```
AccountController.getTransactions()
        ↓
TransactionServiceImpl.getByAccountId()
        ↓
AccountRepository.existsById()
        ↓
TransactionRepository.findByAccountAccountIdOrderByCreatedAtDesc()
        ↓
Page<TransactionResponse>
```

Deposit → ledger (implemented):

```
AccountController.deposit()
        ↓
AccountServiceImpl.deposit()
        ↓
AccountRepository.findById()
        ↓
assert ACTIVE + amount > 0
        ↓
increment availableBalance, ledgerBalance, creditCount, timestamps
        ↓
TransactionService.record(..., CREDIT, ...)
        ↓
TransactionRepository.save()
        ↓
AccountRepository.save()
```

5. Database Tables

`bank_transaction` (entity `Transaction`). FK `account_id` → `account`.

6. REST APIs

Method	Path	Roles
GET	`/accounts/{accountId}/transactions?page&size…`	ADMIN, EMPLOYEE, MANAGER

Hosted on `AccountController` (no separate transaction controller). Planned: `POST /accounts/{id}/withdraw`. Deposit remains on Account API and writes the ledger internally.

7. Controllers

List endpoint on `AccountController`; package `com.bankone.transaction` has no dedicated controller yet.

8. Services

- `TransactionService` / `TransactionServiceImpl.record(...)` — validates account, type, amount > 0, balanceAfter; sets currency from account; persists via repository
- `getByAccountId(accountId, pageable)` — ensures account exists; returns paged `TransactionResponse`

9. Repositories

- `TransactionRepository` extends `JpaRepository<Transaction, Long>`
  - `findByAccountAccountIdOrderByCreatedAtDesc(Long)`
  - `findByAccountAccountIdOrderByCreatedAtDesc(Long, Pageable)`

10. DTOs

`TransactionResponse` for list API. Record path is still internal; deposit still uses Account DTOs.

11. Entities

- `Transaction` → table `bank_transaction`
  - `transactionId`, `account`, `transactionType`, `amount`, `balanceAfter`, `currencyCode`, `narration`, `createdAt`, `createdBy`

12. Utility Classes

None

13. Configuration Classes

None dedicated; deposit and list use existing `/accounts/**` security matchers.

14. Validation Rules

In `record`: account required; type required; amount > 0; balanceAfter required. Deposit path still enforces ACTIVE + amount > 0 before calling `record`.

Planned for debit: sufficient balance; currency match.

15. Security Rules

Write today via Account deposit and openAccount opening deposit (ADMIN/EMPLOYEE). List read aligns with accounts (ADMIN, EMPLOYEE, MANAGER).

16. Exception Handling

`IllegalArgumentException` from `record` / deposit; reuse `GlobalExceptionHandler`

17. Logging

None dedicated yet; planned structured log per posting

18. Audit Events

Ledger row **is** the financial audit (`created_by`, `created_at`, `balance_after`). Optional link to `audit` module for richer who/when metadata later.

19. Testing Strategy

- Deposit creates +1 CREDIT with matching amount and `balance_after`
- Withdraw insufficient funds (when built)
- Concurrent balance safety

20. Future Extension Guide

Next: withdraw (DEBIT); then transfer + beneficiary; then ledger UI; wire dashboard `todayTransactionCount`.

Future Modification Guide

Requirement: Expose transaction list API — done (API)

Item	Detail
Files	`AccountController.java`, `TransactionService` / `Impl`, `TransactionResponse.java`
Classes	`AccountController`, `TransactionServiceImpl`
Methods	`getTransactions`, `getByAccountId`
Impact	Account detail UI still pending; API docs updated

Requirement: Write CREDIT on openAccount

Item	Detail
Files	`AccountServiceImpl.java`
Classes	`AccountServiceImpl`
Methods	`openAccount()` — after save call `transactionService.record(..., CREDIT, ...)`
Impact	Opening balances appear in ledger; dashboard counts

Requirement: Add withdraw (DEBIT)

Item	Detail
Files	`AccountController`, `AccountServiceImpl`, possibly `TransactionService`
Methods	New `withdraw()` — balance check then `record(..., DEBIT, ...)`
Impact	API docs; Accounts UI; status/min-balance rules

Call hierarchy (deposit → record) — implemented

```
AccountController.deposit()
        ↓
AccountServiceImpl.deposit()
        ↓
AccountRepository.findById()
        ↓
TransactionService.record(..., TransactionType.CREDIT, ...)
        ↓
TransactionServiceImpl.record()
        ↓
TransactionRepository.save()
        ↓
AccountRepository.save()
```